Chapter 5: Resampling Methods

MATH 448 — Problems 3, 5, 6, 7

Uzair Hamed Mohammed

October 29, 2026

Problem 3 — k-fold cross-validation

(a) Implementation

Randomly divide the n observations into k roughly equal folds $F_1, ..., F_k$. For each $j = 1, ..., k$, fit the model on the observations outside F_j, predict the observations in F_j, and record the fraction misclassified. The k-fold estimate is

$$\mathrm{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^{k} \frac{1}{|F_j|} \sum_{i \in F_j} 1\Big(y_i \neq \hat{y}_i^{(-j)}\Big),$$

where $\hat{y}_i^{(-j)}$ is the prediction for observation i using the fit that omits fold j.

(b) Comparisons

Validation set. Advantage: only one fit is required and the split is simple. Disadvantage: the estimate has high variance because it depends on which observations happen to land in the validation set, and training on only part of the data tends to overestimate the test error.

LOOCV. Advantage: each fit uses $n - 1$ observations and the procedure has no split randomness, so it has low bias. Disadvantage: it requires n fits, and the n training sets are almost identical, so the estimate can have high variance; k-fold CV often gives a better bias-variance and computational compromise.

Problem 5 — validation-set estimate on Default

A random seed of 0 was fixed before all splits. Each validation split uses 5,000 training and 5,000 validation observations.

(a) Logistic regression fit

Term	Estimate	Std. error	z	p-value
Intercept	-11.5405	0.4348	-26.544	3.04×10^{-155}
income	2.0809×10^{-5}	4.9852×10^{-6}	4.174	2.99×10^{-5}
balance	0.0056	2.2738×10^{-4}	24.835	3.72×10^{-136}

Both income and balance are statistically significant, and both coefficients are positive: higher income and higher balance are associated with a higher probability of default.

(b)–(c) Validation-set error

The first split gives

$$\widehat{\mathrm{Err}}_{\mathrm{val}} = \frac{145}{5000} = 2.90\%.$$

The implementation for one split is:

```
train, valid = train_test_split(np.arange(n), test_size=0.5,
                                random_state=seed)
X = sm.add_constant(Default[["income", "balance"]])
fit = sm.GLM(y.iloc[train], X.iloc[train],
             family=sm.families.Binomial()).fit()
prob = fit.predict(X.iloc[valid])
error = np.mean((prob > 0.5).astype(int) != y.iloc[valid].to_numpy())
```

Repeating the procedure for the three splits:

Split	Errors	Validation size	Validation error	Accuracy
Run 1	145	5000	2.90%	97.10%
Run 2	125	5000	2.50%	97.50%
Run 3	124	5000	2.48%	97.52%
Mean			2.63%	97.37%

The validation error ranges from 2.48% to 2.90%, with mean 2.63% and standard deviation 0.24 percentage points. The estimate is therefore somewhat sensitive to the particular training/validation split, which is the main drawback of the validation-set approach.

(d) Adding the student dummy

Model	Run 1	Run 2	Run 3	Mean
income + balance	2.90%	2.50%	2.48%	2.63%
income + balance + student	2.92%	2.62%	2.54%	2.69%

Including the student dummy does not reduce the validation error: it is higher in all three splits, by 0.02 to 0.12 percentage points (mean increase 0.07 percentage points). The student coefficient is statistically significant and negative after adjusting for income and balance, but this does not translate into better out-of-sample classification.

Problem 6 — bootstrap standard errors

(a) Standard errors from the logistic fit

Predictor	Estimate	Std. error
income	2.0809×10^{-5}	4.9852×10^{-6}
balance	0.0056	2.2738×10^{-4}

(b) Bootstrap function

For a bootstrap index vector $I^* = (i_1, ..., i_n)$, define

$$\text{boot_fn}\,(I^*) = \left(\hat{\beta}^*_{\text{income}}, \hat{\beta}^*_{\text{balance}}\right),$$

the two slope estimates from the logistic regression fitted to the resampled observations $\left(x_{i_j}, y_{i_j}\right)$ for $j = 1, ..., n$:

```
def boot_fn(data, index):
    sample = data.iloc[index]
    y = (sample["default"] == "Yes").astype(int)
    X = sm.add_constant(sample[["income", "balance"]])
    fit = sm.GLM(y, X, family=sm.families.Binomial()).fit()
    return {"income": float(fit.params["income"]),
            "balance": float(fit.params["balance"])}
```

(c) Bootstrap standard errors

Using 1000 bootstrap samples, the estimated standard errors are the sample standard deviations of the bootstrap coefficient values:

Predictor	Logistic (GLM) SE	Bootstrap SE
income	4.9852×10^{-6}	4.7696×10^{-6}
balance	2.2738×10^{-4}	2.3055×10^{-4}

(d) Comparison

The two sets of standard errors agree closely. For income, the bootstrap standard error is 4.77×10^{-6} versus 4.99×10^{-6} from the logistic fit; for balance, it is 2.31×10^{-4} versus 2.27×10^{-4}. The bootstrap therefore supports the model-based standard errors, with only small simulation differences.

Problem 7 — manual LOOCV on Weekly

(a) Full-data fit

Term	Estimate	Std. error	z	p-value
Intercept	0.2212	0.0615	3.599	3.1927×10^{-4}
Lag1	-0.0387	0.0262	-1.477	0.1397
Lag2	0.0602	0.0265	2.270	0.0232

At the 5% level, only Lag2 is significant; its positive coefficient indicates that a higher return two weeks ago is associated with a higher probability of an Up week.

(b)–(c) Fit without the first observation

Excluding the first observation gives $\hat{\beta}_{\text{Lag1}} = \text{-}0.0384$ and $\hat{\beta}_{\text{Lag2}} = 0.0608$, nearly identical to the full-data fit. For the first observation,

$$\hat{P}(Y = \text{Up} \mid X_1) = 0.5714 > 0.5,$$

so the model predicts Up, while the observed direction is Down. The first observation is therefore misclassified.

(d) Leave-one-out loop

For $i = 1, \ldots, n$, perform the following steps:

1. Fit the logistic model using all observations except i.
2. Compute the fitted probability $\hat{P}(Y_i = \text{Up} \mid X_i)$ from that model.
3. Predict $\hat{y}_i = \text{Up}$ if $\hat{P}(Y_i = \text{Up} \mid X_i) > 0.5$, and $\hat{y}_i = \text{Down}$ otherwise.
4. Set $e_i = 1$ if $\hat{y}_i \neq y_i$, and $e_i = 0$ otherwise.

The corresponding loop is:

```
X = sm.add_constant(Weekly[["Lag1", "Lag2"]])
errors = []
for i in range(n):
    train = np.arange(n) != i
    fit = sm.GLM(y_weekly[train], X[train],
                 family=sm.families.Binomial()).fit()
    p = float(fit.predict(X.iloc[[i]]).iloc[0])
    errors.append(int((p > 0.5) != bool(y_weekly.iloc[i])))
loocv_error = np.mean(errors)
```

(e) LOOCV estimate

Averaging the 1089 indicators gives

$$\widehat{\text{Err}}_{\text{LOOCV}} = \frac{490}{1089} = 45.00\%.$$

The corresponding accuracy is 55.00%, which is slightly below the 55.56% accuracy obtained by always predicting Up. Lag1 and Lag2 thus provide essentially no useful out-of-sample signal for the direction of the market.